

A Project Report

Dungeon Exploration

A 2D procedural action-adventure game built with Python and Pygame

Prepared from the Dungeon Exploration project source, the documentation sample, and the supplied Dungeon Quest / Dungeon Exploration reports.

September 2026

Abstract

Dungeon Exploration is a desktop 2D action-adventure game in which the player creates or selects an explorer profile, chooses a class, enters a procedurally generated dungeon, fights monsters, collects keys and treasures, uses consumable items, and either clears the level or records a failed run. The project uses Python, Pygame, JSON profile persistence, grid-based navigation, and a hybrid procedural generation pipeline. The AI portion is primarily a decision and generation system: a genetic algorithm evolves candidate layouts, cellular automata smooth them, breadth-first search measures reachability and repairs disconnected regions, A* validates paths, and a decision-tree-style profile analysis adjusts future monster counts. This report follows the structure of the supplied sample documentation while documenting the implementation that is actually present in this repository.

Acknowledgement

This report was assembled from the project source and the supplied reference documents. It is intended to support project evaluation and presentation of the current implementation.

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
BFS	Breadth-First Search
A*	A-star pathfinding
JSON	JavaScript Object Notation
UI	User Interface
FPS	Frames Per Second

Contents

1 Introduction

2 Literature Review

3 Methodology

4 System Analysis

5 System Design and Internal Flow

6 AI and Procedural Generation

7 System Development and Implementation

8 Testing and Debugging

9 Conclusion and Future Work

1 Introduction

1.1 Overview

The game is a single-process Pygame application with a screen-manager architecture. The player starts at a setup screen, enters a menu, and begins a run. Each run receives a fresh random seed and a new world generated from the active profile. The dungeon is a grid of tiles containing walls, floor, water, lava, traps, treasures, keys, scrolls, lights, decorations, and monsters.

1.2 Problem Statement

A fixed dungeon quickly becomes predictable. The project addresses this by generating a different playable layout for each run while preserving a coherent objective: explore, collect the door key, survive enemies, and reach the locked door. The game also needs to remember progression without requiring a server or database.

1.3 Objectives

- Provide a playable 2D dungeon exploration loop.
- Generate connected and varied dungeons at runtime.
- Support profile creation, class selection, inventory, progression, and persistent statistics.
- Use pathfinding and reachability checks to keep entities and objectives accessible.
- Adapt future dungeon difficulty using recent player outcomes.

1.4 Features

- Explorer profile creation and local JSON persistence.
- Screen flow for setup, menu, game, inventory, profile, pause, and completion/death states.
- Keyboard movement, sprinting, terrain speed effects, mouse aiming, projectiles, and monster combat.
- Keys, treasures, scrolls, coins, bow/light rewards, traps, water, lava, and locked-door progression.
- Scheduled monster spawning and run-history recording.

1.5 Significance

The project demonstrates how game programming, procedural content generation, search algorithms, persistence, UI state management, and adaptive difficulty can be combined into one small but complete interactive system.

1.6 Scope and Limitations

The current implementation is a local desktop game. The ClientSession name is retained from the project design, but persistence is file-based through ``src/client/data/profiles.json``; there is no multiplayer server, SQL database, or online account service. The AI is rule-based and algorithmic rather than a trained machine-learning model. Monster movement and combat behavior are implemented in the game screen/entity layer, while the most significant AI contribution is map generation and profile-aware difficulty selection.

2 Literature Review

2.1 Shattered Pixel Dungeon

This reference demonstrates the replay value of procedural dungeon levels, class choices, resource management, and grid-based exploration. Dungeon Exploration adopts the replayability principle while using real-time Pygame controls and a custom hybrid generator.

2.2 Elona+

Elona+ illustrates the depth possible through classes, exploration, items, and emergent combinations. The current project uses a smaller scope: class statistics, inventory rewards, consumable effects, and persistent profile progression.

2.3 Multiplayer Dungeon Quest

The supplied report identifies static maps and limited item variety as common limitations. This project responds with runtime map generation, traps, liquids, multiple reward types, scheduled enemies, and profile-aware map scoring. It remains a local single-player implementation.

2.4 Related Game Design Lessons

Reference idea	Applied project response
Procedural replayability	Genetic layout population, crossover, mutation, and randomized carving
Resource pressure	Health, stamina, terrain effects, consumables, keys, and coins
Readable objectives	Farthest-reachable door key, locked door, guarded treasure
Adaptive challenge	Recent-game rules adjust the monster limit

3 Methodology

3.1 Software Development Cycle

An iterative SDLC approach fits this project because gameplay features can be implemented and tested in small slices: screen flow, profile persistence, player movement, generation, entity placement, combat, and progression. The supplied sample documentation also recommends an iterative model for evolving requirements and early playable prototypes.

3.2 Technologies and Tools

Technology	Use in the project
Python 3	Application logic, data structures, algorithms, and persistence
Pygame 2.5.8	Window, input, rendering, timing, sprites, and audio-ready game loop
JSON	Local player profiles and progression storage
VS Code	Development and debugging environment
ReportLab / python-pptx	Generation of this deliverable

3.3 Main Modules

- ``main.py``: adds ``src/`` and ``src/client/`` to the import path and starts the application.
- ``src/client/main.py``: initializes Pygame, registers screens, owns the main loop, and shuts down persistence.
- ``src/client/session.py``: loads/saves profiles, registers/selects explorers, records runs, and commits persistent inventory.
- ``src/shared/mapgen/``: contains constants, layout, genetic algorithm, BFS, A*, placement, decision logic, and world generation.
- ``src/client/screens/game_screen.py``: connects generated worlds to player movement, interactions, combat, spawning, and outcomes.

4 System Analysis

4.1 Requirement Analysis

ID	Functional requirement	Implementation evidence
FR1	Create or select an explorer profile	SetupScreen and ClientSession
FR2	Navigate through a menu and select a class	MenuScreen, profile state, config classes
FR3	Generate a playable dungeon	shared.mapgen.generator.generate_map
FR4	Move, sprint, fight, and interact	GameScreen and Player
FR5	Collect items and unlock progression	placement, inventory, and world interaction methods
FR6	Persist progression and recent outcomes	profiles.json and ClientSession
FR7	Adapt future challenge	decision_tree.monster_limit_for_profile

4.2 Non-functional Requirements

- Playability: generation repairs connectivity and validates paths before placing important entities.
- Maintainability: map generation is separated into focused modules instead of one monolithic function.
- Responsiveness: the game loop uses a clock delta and forwards update/draw/event calls to one active screen.
- Reproducibility: generation accepts a seed, allowing a world to be recreated for debugging.
- Portability: the project is Python-based and uses local assets and JSON rather than a required external service.

4.3 Feasibility

Technical feasibility is high because the project uses a small dependency set and standard grid algorithms. Operational feasibility is supported by direct controls and a clear menu-to-game flow. Economic feasibility is favorable for an academic project because the software stack is open source and the game uses local storage. Schedule risk is concentrated in art assets, balancing, and edge cases in gameplay rather than in external infrastructure.

5 System Design and Internal Flow

5.1 Overall Game Flow

Launch -> initialize Pygame and ClientSession -> SetupScreen creates/selects profile -> MenuScreen -> Start Game -> GameScreen generates a seeded world -> player explores and fights -> collect key and objective items -> unlock door and clear level OR health reaches zero -> record run outcome -> retain persistent progress -> return to menu or restart.

5.2 Runtime Loop

- `clock.tick(FPS)` produces frame delta time.
- Pygame events are sent to the active screen.
- The active screen updates player state, world spawn timers, monsters, projectiles, effects, and profile events.
- The active screen draws the world, entities, player, HUD, and overlays.
- `pg.display.flip()` presents the completed frame.

5.3 Screen Architecture

Screen	Responsibility
SetupScreen	Create/select profile and copy snapshot into shared game state
MenuScreen	Start a run, open inventory/profile, or exit
GameScreen	Generate world, run gameplay, interactions, combat, and outcomes
InventoryScreen	Show and use persistent/run items
PauseScreen	Suspend or resume the active game
ProfileScreen	Show player identity and progression
MenuInventoryScreen	Inspect inventory from the menu

5.4 Internal Data Flow

The shared `game_state` dictionary carries player identity, class, inventory, equipped items, progression, temporary run items, kill counts, and run flags between screens. `ClientSession` keeps the durable copy in `profiles.json`. A generated `GeneratedWorld` carries tiles, spawn, door key, door, treasures, keys, scrolls, monsters, decorations, scheduled spawns, and reachable coordinates into `GameScreen`.

5.5 Objective and Interaction Flow

The placement module finds a farthest reachable tile for the door key, then another farthest tile for the door. Treasures are placed on reachable locations and keys are placed away from their corresponding treasures. During play, the player resolves nearby world interactions, collects items, opens treasures when the required key exists, and changes the door from locked to open when the door key condition is satisfied.

6 AI and Procedural Generation

6.1 AI Definition in This Project

The project does not train a neural network. Its AI is an explainable algorithmic system that makes generation and difficulty decisions from rules, search results, metrics, randomness, and player history. This is appropriate for a procedural game because the desired behavior is controllable, testable, and fast enough to run when a level starts.

6.2 World Generation Pipeline

Stage	Algorithm / module	Purpose
1	genetic_algorithm.generate_population	Create 8 random 81 x 81 seed grids
2	layout.carve_nine_sector_maze	Create a nine-sector maze and connect sector centers
3	cellular_automata.smooth	Smooth local wall/floor patterns
4	bfs.repair_connectivity	Carve paths between disconnected walkable regions
5	bfs.evaluate	Measure open ratio, reachable count, and reachable ratio
6	genetic_algorithm.crossover/mutate	Breed two survivors into the next population
7	decision_tree.choose_best_candidate	Select the highest profile-weighted candidate
8	placement.place_entities	Place objectives, rewards, monsters, lights, and decoration

6.3 Genetic Algorithm

Each generation evaluates candidate maps. The best two grids survive, a crossover combines regions from both parents, and mutation changes individual tiles with a rate that rises by generation: 0.06, 0.08, 0.10, and 0.12 under the current defaults. Four generations are processed. The algorithm is not optimizing a single visual score: it balances walkability, openness, reachable area, and profile statistics through ``score_candidate``.

6.4 Search Algorithms

BFS starts at the selected spawn and visits four-directional walkable neighbors. It supplies reachability and distance maps, finds farthest objective locations, and repairs disconnected regions. A* uses Manhattan distance as its heuristic and checks whether a monster or entity has a valid route to the player. These checks prevent important content from being placed in inaccessible regions.

6.5 Profile-aware Decision Tree

``mine_recent_rules`` reads at most the last five games and converts each into a compact transaction containing outcome, performance, and collected items. It estimates two associations: high kills -> clear and low kills -> loss. ``monster_limit_for_profile`` then applies transparent rules: two or more consecutive losses plus a strong low-kill/loss association increases the next monster limit up to 18; a strong high-kill/clear association increases it modestly; otherwise the default is 12. This creates adaptive difficulty without hiding the reason for the change.

6.6 Entity Placement Strategy

- Objectives use farthest reachable positions to encourage exploration.
- Treasures receive randomized rewards such as coins, bow, or light.
- Keys are placed away from their target treasures, creating navigation pressure.
- Monsters guard treasures and are also scheduled to appear later in the run.
- Wall lights and liquid clusters add environmental variety after functional placement.

7 System Development and Implementation

7.1 Programming Platform

Python is used for application logic, game state, algorithms, and persistence. Pygame provides the event loop, display, timing, drawing, image loading, input, and sprite support. JSON stores profiles locally. The application starts at the root `main.py` and imports the client package after adding the source directories to `sys.path`.

7.2 Hardware and Software Requirements

Category	Recommended baseline
Operating system	Windows, Linux, or macOS
Runtime	Python 3 with Pygame 2.5.8
Memory	4 GB RAM or more
Storage	Project files plus image assets
Display	A desktop resolution supported by <code>config.py</code>
Development	VS Code recommended

7.3 Profile and Run Persistence

Registration creates a default player record. Selection loads the snapshot into `game_state`. At the beginning of a run, the game stores a baseline profile and inventory. On clear or death, `ClientSession` records the result, keeps only the five most recent games for the adaptive rules, removes temporary keys, and commits persistent items, coins, and item usage.

7.4 Important Implementation Note

The repository also contains an older `src/shared/tilemap.py` implementation and a separate `demo/` pipeline. The running client imports `shared.mapgen.generate_map`, so this report treats `src/shared/mapgen/` as the authoritative active implementation and describes the other path as legacy/demo material.

8 Testing and Debugging

8.1 Testing Approach

Testing should combine module checks, deterministic seeded generation, gameplay smoke tests, and persistence checks. A fixed seed makes a generated map reproducible while random seeds verify variety. The most valuable assertions are connectivity, reachable objectives, valid monster paths, and correct run-history updates.

8.2 Test Cases

ID	Test	Expected result
T1	Launch root `main.py`	Pygame window opens at SetupScreen
T2	Create explorer with a name	Profile is saved and menu opens
T3	Select existing explorer	Snapshot restores identity, items, and profile
T4	Start game with fixed seed	World is generated with a valid spawn
T5	Inspect reachable set	Important regions are connected after repair
T6	Check door key and door	Both are reachable and placed separately
T7	Move into walls and liquids	Walls block; terrain changes speed/effects
T8	Attack a monster	Projectile and health/counter logic update
T9	Wait for scheduled spawn	Monster enters active list at its spawn time
T10	Clear or lose a run	Outcome is recorded and temporary keys are removed
T11	Use recent losses	Next map monster limit follows decision rules
T12	Run syntax compilation	Python source compiles without syntax errors

8.3 Current Verification

The deliverable generator itself is executable in the configured project virtual environment. The project source can be syntax-compiled independently. A full interactive playtest requires the Pygame dependency and a graphical session; this report does not claim a completed GUI playthrough.

9 Conclusion and Future Work

Dungeon Exploration combines a clear game loop with a technically interesting procedural generation system. The central design strength is the layered AI pipeline: generation produces candidates, search algorithms enforce playability, placement creates navigable objectives, and recent-history rules adapt enemy quantity. The screen architecture and local profile store provide a practical foundation for continued gameplay work.

Future improvements

- Add automated tests for generator invariants and decision-tree boundary cases.
- Expose a debug seed and generation metrics in a developer overlay.
- Improve monster behavior with line-of-sight, pursuit states, attack range, and difficulty parameters.
- Add audio, richer class-specific abilities, more level objectives, and boss encounters.
- Separate gameplay services from rendering further and remove legacy duplicate map-generation code.
- Add optional multiplayer only after the single-player rules and persistence model are stable.

References

- Project source: ``src/client/main.py``, ``src/client/session.py``, ``src/client/screens/game_screen.py``.
- Project source: ``src/shared/mapgen/generator.py``, ``genetic_algorithm.py``, ``layout.py``, ``bfs.py``, ``astar.py``, ``decision_tree.py``, and ``placement.py``.
- Supplied reference: ``documentation/documentation_sample.docx``.
- Supplied reference: ``documentation/dungeon_exploration.docx``.
- Supplied reference: ``documentation/dungeon_quest (Repaired).docx``.
- Pygame documentation: <https://www.pygame.org/docs/>.